COURSE: WEB SECURITY & PENETRATION TESTING

LAB ASSIGNMENT REPORT: DAY 43 BOLA VULNERABILITIES

Student Name: Jiwosh Shrestha

Topic: Broken Object Level Authorization (BOLA)

Assignment: Day 43 - BOLA

Focus: OWASP API1:2023 - Broken Object Level Authorization

PART 1: BOLA FUNDAMENTALS & AUTHORIZATION CONCEPTS

Question 1: Define: Authentication, Authorization, Object-Level Authorization.

• Authentication: The security mechanism verifying entity identity ('Who are you?').
• Authorization: The permission check determining actions allowed for an authenticated user ('What are you permitted to do?').
• Object-Level Authorization: The specific access control check verifying that the current authenticated user has explicit permission to view, modify, or delete the specific object instance identified in the request (e.g., verifying User A owns Order #42).

Question 2: Explain Broken Object Level Authorization (BOLA). Why is the presence of an object ID alone not enough to prove BOLA?

BOLA occurs when an API accepts a client-supplied object identifier and processes the request without validating whether the caller possesses authorized ownership over that specific object.

The presence of an object ID in a request URI or body is standard REST API design. BOLA is proven only when substituting one user's object ID with another user's ID succeeds, returning data or performing state changes that the requesting user should not access.

Question 3: Explain the difference between BOLA and Broken Function Level Authorization (BFLA).

• BOLA (Horizontal Escalation): Focuses on unauthorized access across object instances within the same privilege tier (e.g., User A accessing User B's private user record).
• BFLA (Vertical Escalation): Focuses on unauthorized access to administrative functions or endpoints restricted to higher-tier roles (e.g., a standard user calling POST /admin/deleteUser).

Question 4: Give four places where an API object identifier may appear in a request.

1. Path Parameter: /api/orders/42
2. Query Parameter: ?orderId=42
3. Request Body Property: {"orderId": 42}
4. Custom HTTP Header: X-Resource-Id: 42

Question 5: Explain why replacing sequential numeric IDs with UUIDs does NOT fix broken object-level authorization.

UUIDs only obscure identifiers making sequential guessing (enumeration) harder; they do not enforce backend authorization rules. If the server returns any object whose UUID is supplied without validating ownership against the user's session token, an attacker who obtains a valid UUID (via logs, referrers, or data leaks) can still bypass access controls.

Access control validation must occur on every request regardless of whether IDs are sequential integers or random UUIDs.

PART 2: PRACTICAL FINDINGS & crAPI BOLA PROOF-OF-CONCEPT

Question 6: Controlled crAPI BOLA Finding & Evidence Log

Lab Execution Methodology:
• Baseline Request: GET /identity/api/v2/user/dashboard -> 200 OK (User authenticated, own profile loaded).
• Object Reference Identified: vehicleId / userId present in path parameter.
• Cross-Object Injection Test: Substituted user's own ID with victim user ID in Burp Repeater.

```
GET /identity/api/v2/vehicle/[victim-user-id]/location HTTP/1.1
Host: crapi.local
Authorization: Bearer [REDACTED_ATTACKER_TOKEN]

HTTP/1.1 200 OK
Content-Type: application/json

{
  "carId": "victim-car-99",
  "location": { "lat": "37.7749", "long": "-122.4194" },
  "ownerEmail": "victim@example.com"
}
```

Professional Analysis:
• Expected Secure Behavior: The server rejects the cross-object request with HTTP 403 Forbidden (or 404 Not Found), restricting access strictly to the true vehicle owner.
• Observed Vulnerable Behavior: The server returned HTTP 200 OK containing victim location data when the object ID was modified—confirming BOLA.

PART 3: SECURE DESIGN RECOMMENDATIONS

Question 7: Give at least six recommendations for preventing BOLA.

1. Mandatory Server-Side Validation: Validate object ownership on every incoming request containing an object ID.
2. Session-Based Scope Derivation: Derive authorized resource IDs directly from user session context rather than trusting client parameters.
3. Indirect Object References: Utilize session-mapped indirect references or tokenized temporary resource maps.
4. Safe Failure Status Codes: Return HTTP 403 Forbidden or generic 404 Not Found without leaking resource existence.
5. Automated Security Regression Testing: Integrate object-ownership authorization tests into CI/CD pipelines.
6. Security Monitoring & Anomaly Detection: Monitor and alert on sudden spikes in cross-object access attempts from single source IPs.
7. Opaque Identifiers as Defense-in-Depth: Combine UUIDs with strong authorization checks as a defense-in-depth measure.